

Resumen Teórico: Reducciones y Reducibilidad

Computabilidad y Complejidad — UNRC

Basado en el material de Pablo Castro (UNRC)

Índice

1. Introducción	1
2. La idea de reducción	1
2.1. Esquema general	1
3. Ejemplos clásicos de reducciones	2
3.1. El problema de terminación: H_{TM} es indecidible	2
3.2. Vacío del lenguaje: E_{TM} es indecidible	2
3.3. Reg es indecidible	3
3.4. Equivalencia de MTs: Eq_{TM} es indecidible	3
4. El Teorema de Rice	4
4.1. Propiedades de MTs	4
4.2. El teorema	4
5. Historias de ejecución y autómatas linealmente acotados	5
5.1. Historia de ejecución	6
5.2. Autómatas linealmente acotados (LBAs)	6
5.3. A_{LBA} es decidable	6
5.4. E_{LBA} es indecidible	7
6. El Problema de Correspondencia de Post	7
6.1. Definición y ejemplo	7
6.2. PCP es indecidible	8
7. La noción formal de reducibilidad	9
7.1. Funciones computables	9
7.2. Reducción de mapeo	9
7.3. Propiedades fundamentales	9
7.4. Contrapositivas: la herramienta para indecidibilidad	10
7.5. Ejemplo formal: $A_{TM} \leq Halt$	10
8. Resumen final	11

1. Introducción

En la teoría anterior vimos por diagonalización que existen problemas *indecidibles*, siendo el ejemplo paradigmático el Halting Problem A_{TM} . La diagonalización es una técnica poderosa,

pero también delicada: construir un nuevo argumento diagonal cada vez que queremos probar la indecidibilidad de un problema sería trabajoso y propenso a errores.

La técnica de **reducciones** permite, en cambio, demostrar la indecidibilidad de un problema B *reutilizando* la indecidibilidad de un problema A que ya conocemos:

“Si pudiéramos resolver B , entonces podríamos resolver A . Pero sabemos que A es irresoluble, así que B también lo es.”

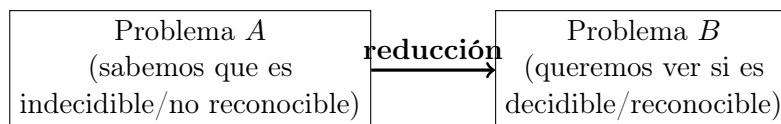
En este resumen vamos a:

1. Ver la idea informal de *reducir un problema a otro* y aplicarla a varios ejemplos clásicos: H_{TM} , E_{TM} , Reg , Eq_{TM} .
2. Demostrar el **Teorema de Rice**: toda propiedad no trivial del lenguaje de una MT es indecidible.
3. Introducir las *historias de ejecución* y los **autómatas linealmente acotados** (LBAs).
4. Presentar el **Problema de Correspondencia de Post** (PCP) y mostrar que es indecidible.
5. Formalizar la noción de **reducción** mediante funciones computables, y derivar las propiedades fundamentales: la reducción *preserva decidibilidad* y *preserva reconocibilidad*.

2. La idea de reducción

2.1. Esquema general

Cuando queremos demostrar que un problema B es **indecidible**, la estrategia general es:



La idea es: *si pudiéramos resolver B , entonces podríamos resolver A* . Como sabemos que A no se puede resolver, B tampoco. La reducción suele construirse usando una máquina de Turing.

Intuición

La dirección de la implicación es crucial. Para probar que B es *indecidible*, reducimos un problema *conocido como indecidible* A a B (escribiremos esto luego como $A \leq B$). *Nunca al revés*: que un problema decidible se pueda reducir a B no aporta información.

3. Ejemplos clásicos de reducciones

3.1. El problema de terminación: H_{TM} es indecidible

Sea

$$H_{TM} = \{\langle M, w \rangle \mid M \text{ termina con la entrada } w\}.$$

Este lenguaje, conocido como “el problema de terminación”, es el problema de predecir si una MT va a terminar (sin importar si acepta o rechaza).

Teorema 3.1. H_{TM} es indecidible.

Prueba (reducción desde A_{TM}). Supongamos por absurdo que existe una MT H que decide H_{TM} . Construimos una MT M' que decidiría A_{TM} :

Sobre la entrada $\langle M, w \rangle$, M' procede así:

1. Ejecuta H con la entrada $\langle M, w \rangle$.
 - Si H rechaza (es decir, M no termina con w), M' **rechaza**.
 - Si H acepta (es decir, M sí termina con w), seguimos al paso 2.
2. Simula M con la entrada w . Como en este punto sabemos que M termina, la simulación termina:
 - Si M acepta, M' acepta.
 - Si M rechaza, M' rechaza.

M' siempre termina (porque la simulación del paso 2 solo se hace si H garantiza terminación) y acepta exactamente cuando M acepta w . Por lo tanto M' decide A_{TM} , lo cual es una contradicción con la indecidibilidad de A_{TM} . $\square$

Intuición

Sin un decididor de la terminación no podemos simular ciegamente, porque la simulación podría no terminar. Pero *con* ese decididor, primero le preguntamos “¿esto termina?”, y solo si la respuesta es Sí seguimos. Así reducimos A_{TM} a H_{TM} .

3.2. Vacío del lenguaje: E_{TM} es indecidible

Sea

$$E_{TM} = \{\langle M \rangle \mid \mathcal{L}(M) = \emptyset\}.$$

Teorema 3.2. E_{TM} es indecidible.

Prueba (reducción desde A_{TM}). Supongamos que existe una MT E que decide E_{TM} . Construimos una MT M' que decidiría A_{TM} .

Sobre la entrada $\langle M, w \rangle$, M' procede así:

1. Construye una nueva MT S que, dada una entrada x , hace lo siguiente:
 - Si $x \neq w$, rechaza.
 - Si $x = w$, simula M con la entrada w y responde lo que responda M .
2. Corre E con la entrada $\langle S \rangle$.
 - Si E acepta (es decir, $\mathcal{L}(S) = \emptyset$), entonces M no acepta w y M' rechaza.
 - Si E rechaza (es decir, $\mathcal{L}(S) \neq \emptyset$), entonces M acepta w y M' acepta.

¿Por qué funciona? Por construcción, $\mathcal{L}(S)$ es vacío si M no acepta w , y $\mathcal{L}(S) = \{w\}$ si M acepta w . Detectar el vacío de $\mathcal{L}(S)$ es entonces equivalente a decidir si M acepta w . Esto contradice la indecidibilidad de A_{TM} . $\square$

Idea clave

Esta técnica es muy general: *filtrar* la entrada para que la máquina construida S “haga algo interesante” solo en una cadena particular, y luego usar al supuesto decididor del problema B para extraer la información que necesitamos.

3.3. Reg es indecidible

Sea

$$Reg = \{\langle M \rangle \mid \mathcal{L}(M) \text{ es un lenguaje regular}\}.$$

Teorema 3.3. Reg es indecidible.

Prueba (reducción desde A_{TM}). Supongamos que R decide Reg . Construimos N que decidiría A_{TM} .

Sobre la entrada $\langle M, w \rangle$, N procede así:

1. Construye una nueva MT M' que, dada una entrada x :
 - Si $x \in \{0^n 1^n \mid n \geq 0\}$, acepta directamente.
 - Si $x \notin \{0^n 1^n \mid n \geq 0\}$, simula M con la entrada w y acepta si M acepta.
2. Corre R con la entrada $\langle M' \rangle$. Devuelve lo mismo que R .

¿Por qué funciona? Analicemos qué lenguaje reconoce M' :

- Si M acepta w : M' acepta toda cadena (las de $\{0^n 1^n\}$ por la rama 1, y las demás porque la simulación del paso 2 acepta). Es decir $\mathcal{L}(M') = \Sigma^*$, que es regular.
- Si M no acepta w : M' acepta exactamente $\{0^n 1^n \mid n \geq 0\}$, que es *no regular*.

Por lo tanto, $\mathcal{L}(M')$ es regular si y solo si M acepta w . Si pudiéramos decidir Reg (con la máquina R), podríamos decidir A_{TM} . Contradicción. $\square$

3.4. Equivalencia de MTs: Eq_{TM} es indecidible

Sea

$$Eq_{TM} = \{\langle M, M' \rangle \mid \mathcal{L}(M) = \mathcal{L}(M')\}.$$

Teorema 3.4. Eq_{TM} es indecidible.

Prueba (reducción desde E_{TM}). Supongamos que R decide Eq_{TM} . Construimos E que decidiría E_{TM} .

Sobre la entrada $\langle M \rangle$, E procede así:

1. Sea M_0 una MT fija que rechaza todas las cadenas (es decir, $\mathcal{L}(M_0) = \emptyset$).
2. Corre R con la entrada $\langle M, M_0 \rangle$. Devuelve lo mismo que R .

¿Por qué funciona? R acepta sii $\mathcal{L}(M) = \mathcal{L}(M_0) = \emptyset$, es decir, exactamente cuando $\langle M \rangle \in E_{TM}$. Como E_{TM} es indecidible (ya lo probamos arriba), esto da una contradicción. $\square$

4. El Teorema de Rice

Después de varias reducciones similares, surge una pregunta natural: ¿hay alguna propiedad “interesante” del lenguaje de una MT que sí sea decidible? La respuesta, debida a Henry Rice, es notable: **no**. Cualquier propiedad no trivial del lenguaje aceptado por una MT es indecidible.

4.1. Propiedades de MTs

Definición 4.1 (Propiedad de lenguajes de MTs). Un conjunto $P \subseteq \{\langle M \rangle \mid M \text{ es una MT}\}$ se dice una **propiedad** si *depende únicamente del lenguaje reconocido*: si $\mathcal{L}(M) = \mathcal{L}(M')$, entonces

$$\langle M \rangle \in P \iff \langle M' \rangle \in P.$$

Definición 4.2 (Propiedad no trivial). Una propiedad P se dice **no trivial** si:

1. $P \neq \emptyset$ (alguna MT tiene la propiedad), y
2. $P \neq \{\langle M \rangle \mid M \text{ es una MT}\}$ (no *todas* las MTs la tienen).

Ejemplo 4.3. Son ejemplos de propiedades no triviales:

- “ $\mathcal{L}(M) = \emptyset$ ” (vacío).
- “ $\mathcal{L}(M)$ es regular”.
- “ $\mathcal{L}(M)$ contiene una cadena específica w_0 ”.
- “ $\mathcal{L}(M) = \Sigma^*$ ”.

Las propiedades *triviales* (todas o ninguna) son: “ $\mathcal{L}(M)$ es cualquier lenguaje” y “ $\mathcal{L}(M)$ es imposible”.

4.2. El teorema

Teorema 4.4 (Rice). Toda propiedad no trivial sobre lenguajes de máquinas de Turing es indecidible.

Prueba. Sea P una propiedad no trivial. Supongamos por absurdo que existe una MT M_P que decide P . Vamos a reducir A_{TM} a P , llegando a una contradicción.

Paso 1: elegir un representante de P con lenguaje no vacío. Como P es no trivial, hay alguna MT que pertenece a P . Distinguiamos dos casos:

- Si la MT “trivial” $M_\emptyset$ con $\mathcal{L}(M_\emptyset) = \emptyset$ no está en P : elegimos cualquier $T \in P$. Como $T \in P$ y $M_\emptyset \notin P$, debe tenerse $\mathcal{L}(T) \neq \emptyset$.
- Si $M_\emptyset \in P$: trabajamos con $\overline{P}$ en lugar de P . Esa propiedad sigue siendo no trivial (por hipótesis P no es la propiedad trivial total) y ahora $M_\emptyset \notin \overline{P}$, así que aplica el caso anterior. Decidir $\overline{P}$ es equivalente a decidir P .

En cualquier caso, suponemos sin pérdida de generalidad que tenemos $T \in P$ con $\mathcal{L}(T) \neq \emptyset$, y $M_\emptyset \notin P$.

Paso 2: construir la reducción. Sobre la entrada $\langle M, w \rangle$ de A_{TM} , construimos la MT M_w que, dada una entrada x , hace lo siguiente:

1. Simula M con la entrada w .
 - Si M rechaza w , M_w rechaza x .

- Si M acepta w , simula T con la entrada x y acepta si T acepta.

Paso 3: analizar el lenguaje de M_w .

- Si M acepta w : para toda x , M_w simula T con x , así que $\mathcal{L}(M_w) = \mathcal{L}(T)$. Por lo tanto $\langle M_w \rangle \in P$.
- Si M no acepta w : M_w rechaza toda x (o no termina, pero nunca acepta), así que $\mathcal{L}(M_w) = \emptyset$. Por lo tanto $\langle M_w \rangle \notin P$ (porque $M_\emptyset \notin P$).

Paso 4: contradicción. Si pudiéramos decidir P , decidiríamos A_{TM} : dado $\langle M, w \rangle$, construimos M_w y le preguntamos a M_P si $\langle M_w \rangle \in P$. Pero A_{TM} es indecidible. Por lo tanto, P no puede ser decidable. $\square$

Idea clave

El Teorema de Rice es una herramienta extremadamente poderosa: una vez probado, podemos demostrar la indecidibilidad de muchísimos problemas con una sola línea: “por Rice”. Basta con verificar que la propiedad sea *no trivial* y que dependa solo del lenguaje.

Ejemplo 4.5. Por el Teorema de Rice, son inmediatamente indecidibles:

- “ $\mathcal{L}(M) = \emptyset$ ”,
- “ $\mathcal{L}(M)$ es finito”,
- “ $\mathcal{L}(M)$ es regular”,
- “ $\mathcal{L}(M)$ contiene a la cadena 0101”.

Observación 4.6. El Teorema de Rice se aplica a propiedades del *lenguaje*. Hay propiedades de la *máquina* (no del lenguaje) que sí son decidibles, por ejemplo: “ M tiene exactamente 5 estados” o “ M usa el alfabeto $\{0, 1\}$ ”. Estas no son propiedades en el sentido de Rice porque dos MTs pueden reconocer el mismo lenguaje y diferir en estas propiedades.

5. Historias de ejecución y autómatas linealmente acotados

Algunas reducciones requieren codificar la *ejecución completa* de una MT, no solo el resultado. Para esto se introduce la noción de *historia de ejecución*.

5.1. Historia de ejecución

Definición 5.1 (Historia de ejecución). Dada una MT M y una entrada w , una **historia de ejecución** es una secuencia de configuraciones

$$C_0, C_1, \dots, C_k$$

tal que:

- $C_0 = q_0 w$ es la configuración inicial.
- Para $0 \leq i < k$, $C_i \rightarrow_M C_{i+1}$ es un paso legal de cómputo de M .
- C_k es una configuración final (acepta o rechaza).

Una MT determinista tiene a lo sumo *una* historia de ejecución para cada entrada. Las historias que terminan en q_A son *aceptantes*; las que terminan en q_R son *rechazantes*.

5.2. Autómatas linealmente acotados (LBAs)

Definición 5.2 (Autómata linealmente acotado). Un **autómata linealmente acotado** (LBA) es una MT que *solo* puede usar la porción de la cinta ocupada por la entrada como memoria. Si el cabezal intenta salir hacia la izquierda o la derecha, se mantiene en la posición límite.

Intuición

Los LBA son un modelo intermedio entre los autómatas finitos y las MTs generales: tienen memoria, pero limitada al tamaño de la entrada. Esto los hace estrictamente más débiles que las MTs pero más fuertes que los autómatas a pila.

Cota sobre la cantidad de configuraciones. Dada una entrada de tamaño n y un LBA con conjunto finito de estados Q y alfabeto de cinta Γ , la cantidad de configuraciones posibles está acotada por

$$|Q| \cdot n \cdot |\Gamma|^n.$$

Esto se debe a que una configuración está determinada por: (i) el estado actual ($|Q|$ opciones), (ii) la posición del cabezal (n opciones), (iii) el contenido de la cinta ($|\Gamma|^n$ opciones, porque hay n celdas y cada una contiene un símbolo de Γ).

5.3. A_{LBA} es decidible

Teorema 5.3. El lenguaje

$$A_{LBA} = \{\langle A, w \rangle \mid A \text{ es un LBA y } A \text{ acepta } w\}$$

es decidible.

Idea. El problema en una MT general es que puede entrar en un bucle infinito sin que podamos detectarlo. Pero en un LBA, la cantidad de configuraciones es *finita* y acotada. Si la simulación se extiende más allá de esa cota, es porque hemos repetido una configuración: estamos en un bucle.

Algoritmo. Sobre la entrada $\langle A, w \rangle$ con $|w| = n$:

1. Simular A con la entrada w durante a lo sumo $|Q| \cdot n \cdot |\Gamma|^n$ pasos.
2. Si en algún momento A acepta, aceptar.
3. Si en algún momento A rechaza, rechazar.
4. Si se superó la cantidad máxima de pasos sin terminar, rechazar (es un bucle).

El algoritmo siempre termina, así que A_{LBA} es decidible. □

5.4. E_{LBA} es indecidible

A pesar de lo anterior, no todo es decidible para LBAs:

Teorema 5.4. El lenguaje

$$E_{LBA} = \{\langle A \rangle \mid A \text{ es un LBA y } \mathcal{L}(A) = \emptyset\}$$

es *indecidible*.

Prueba (idea, reducción desde A_{TM}). Vamos a reducir A_{TM} a E_{LBA} .

Sobre la entrada $\langle M, w \rangle$, construimos un LBA E cuyo lenguaje sea *la codificación de la única historia de ejecución aceptante de M con entrada w* (si existe).

Concretamente, E recibe como entrada una cadena que pretende ser una secuencia $C_0 \# C_1 \# \dots \# C_k$, y chequea:

- Que $C_0 = q_0 w$ sea la configuración inicial correcta.
- Que cada C_{i+1} se obtenga de C_i aplicando un paso legal de M .
- Que C_k contenga el estado q_A .

Todas estas verificaciones se hacen con memoria proporcional al tamaño de la entrada (que es la historia), así que E es efectivamente un LBA.

Observación clave:

- Si M acepta w , hay una única historia aceptante, y E la acepta. Por lo tanto $\mathcal{L}(E) \neq \emptyset$.
- Si M no acepta w , no hay historia aceptante, y $\mathcal{L}(E) = \emptyset$.

Entonces, decidir E_{LBA} permitiría decidir A_{TM} . Como A_{TM} es indecidible, E_{LBA} también lo es. $\square$

6. El Problema de Correspondencia de Post

6.1. Definición y ejemplo

El **Problema de Correspondencia de Post** (PCP) es un problema combinatorio muy sencillo de enunciar pero indecidible. Se trabaja con *fichas de dominó*, cada una con una cadena arriba y otra abajo:

$$\frac{b}{ca}, \quad \frac{a}{ab}, \quad \frac{ca}{a}, \quad \frac{abc}{c}.$$

Pregunta: ¿existe una secuencia de fichas (con posibles repeticiones) tal que la concatenación de las cadenas de arriba sea *igual* a la concatenación de las cadenas de abajo?

Ejemplo 6.1. Para el conjunto anterior, una solución es:

$$\frac{a}{ab}, \frac{b}{ca}, \frac{ca}{a}, \frac{a}{ab}, \frac{abc}{c}.$$

Arriba leemos: $a \cdot b \cdot ca \cdot a \cdot abc = abcaaabc$. Abajo leemos: $ab \cdot ca \cdot a \cdot ab \cdot c = abcaaabc$. ¡Coinciden!

Formalmente, el lenguaje asociado es

$$PCP = \{ \langle D \rangle \mid D \text{ es un conjunto de fichas con solución} \}.$$

6.2. PCP es indecidible

Teorema 6.2 (Post). PCP es indecidible.

Idea de la prueba (reducción desde A_{TM}). La prueba consiste en mostrar que, dado un par $\langle M, w \rangle$, podemos construir un conjunto de fichas D tal que D tiene solución si y solo si M acepta w . La idea es codificar la *historia de ejecución* de M con entrada w mediante el juego de fichas.

Fichas que codifican δ . Para cada transición $\delta(q_i, a) = (q_j, b, R)$ se introduce la ficha

$$\frac{q_i a}{b q_j}.$$

Para cada transición $\delta(q_i, b) = (q_j, c, L)$ se agrega, para cada símbolo a ,

$$\frac{a q_i b}{q_j a c}.$$

Fichas de copia. Para cada símbolo a del alfabeto, se agrega

$$\frac{a}{a},$$

que permite mantener intactos los símbolos no afectados por el cabezal.

Ficha de inicio. Se incluye una ficha especial que “inicializa” la solución con la configuración $\# q_0 w \#$ arriba y abajo desfasados.

Fichas de cierre. Se introducen fichas para que, una vez que aparece el estado de aceptación q_A , las cadenas de arriba y de abajo finalmente se “empaten”.

¿Cómo funciona el juego? En cualquier solución parcial, la cadena de arriba está siempre un paso atrasada respecto de la cadena de abajo: la parte de abajo es siempre la configuración actual de M , y la parte de arriba es la configuración anterior. Las fichas que codifican δ son las únicas que permiten avanzar. Si M acepta w , hay una secuencia finita de configuraciones que termina en q_A , y las fichas de cierre permiten “alcanzar” a la parte de arriba; si M no acepta, las dos cadenas nunca pueden alinearse.

Conclusión. El conjunto D así construido tiene solución sii M acepta w . Por lo tanto, decidir PCP permitiría decidir A_{TM} . $\square$

Intuición

La importancia del PCP es que se trata de un problema *puramente combinatorio* (no menciona máquinas ni cómputo) y aun así es indecidible. Esto significa que la indecidibilidad no es un fenómeno exclusivo de los problemas “sobre programas”; también aparece en problemas matemáticos elementales. PCP es muy útil como “punto de partida” para demostrar indecidibilidad en problemas de áreas no relacionadas con MTs (lenguajes formales, gramáticas, ecuaciones, etc.).

7. La noción formal de reducibilidad

Hasta ahora usamos la palabra “reducción” de forma informal. Vamos a formalizarla en términos de funciones computables.

7.1. Funciones computables

Definición 7.1 (Función computable). Una función $f : \Sigma^* \rightarrow \Sigma^*$ se dice **computable** si existe una MT M que, comenzando con w en la cinta, termina con $f(w)$ en la cinta.

Ejemplo 7.2. Son funciones computables:

- Las funciones aritméticas: dada una entrada $\langle m, n \rangle$ podemos diseñar una MT que termine con $m + n$.

- Las transformaciones entre MTs: dada la descripción $\langle M \rangle$ podemos construir, con una MT, una descripción $\langle M' \rangle$ de otra MT que se comporte de algún modo determinado a partir de M .

Idea clave

Las funciones computables son herramientas para *transformar entradas*. Sirven, por ejemplo, para reescribir una instancia de un problema en una instancia de otro problema, sin perder información sobre la respuesta.

7.2. Reducción de mapeo

Definición 7.3 (Reducibilidad de mapeo). El lenguaje A es **reducible** al lenguaje B , escrito $A \leq B$, si existe una función computable $f : \Sigma^* \rightarrow \Sigma^*$ tal que para toda w ,

$$w \in A \iff f(w) \in B.$$

La función f se llama una **reducción** de A a B .

Intuición

Tener una reducción $A \leq B$ significa que cualquier instancia de A puede transformarse algorítmicamente en una instancia de B que tiene la *misma respuesta*. Por lo tanto, si supiéramos resolver B , podríamos resolver A : traducimos primero la entrada con f y aplicamos la solución de B .

7.3. Propiedades fundamentales

Teorema 7.4 (Preservación de decidibilidad). Si $A \leq B$ y B es decidible, entonces A es decidible.

Prueba. Sea M_B una máquina de decisión para B , y f la reducción de A a B . Construimos una máquina de decisión M_A para A así:

Sobre la entrada w :

1. Computar $f(w)$ (esto siempre termina porque f es computable).
2. Ejecutar M_B sobre la entrada $f(w)$.
3. Devolver la misma respuesta que M_B .

Por la propiedad de la reducción, $w \in A \iff f(w) \in B \iff M_B$ acepta $f(w)$. Como M_B siempre termina, M_A también. Por lo tanto M_A decide A . $\square$

Teorema 7.5 (Preservación de reconocibilidad). Si $A \leq B$ y B es Turing reconocible, entonces A es Turing reconocible.

Prueba. Idéntica a la anterior, pero usando una MT M_B que solo *reconoce* B . La construcción de M_A es la misma:

Sobre la entrada w :

1. Computar $f(w)$.
2. Ejecutar M_B sobre la entrada $f(w)$.

3. Devolver la misma respuesta.

M_A acepta w sii M_B acepta $f(w)$ sii $f(w) \in B$ sii $w \in A$. □

7.4. Contrapositivas: la herramienta para indecidibilidad

Tomando las contrapositivas de los teoremas anteriores obtenemos las versiones *útiles* para probar indecidibilidad/no reconocibilidad:

- Si $A \leq B$ y A es *indecidible*, entonces B es indecidible.
- Si $A \leq B$ y A *no es reconocible*, entonces B no es reconocible.

Idea clave

Esta es la forma estándar de usar reducciones. Para probar que B es indecidible:

1. Tomamos un problema A que sabemos indecidible (por ejemplo, A_{TM}).
2. Construimos una función computable f con $w \in A \iff f(w) \in B$.

La contrapositiva del teorema nos garantiza que B es indecidible.

7.5. Ejemplo formal: $A_{TM} \leq Halt$

Veamos cómo aplicar la definición formal al problema de terminación. Recordemos

$$Halt = H_{TM} = \{\langle M, w \rangle \mid M \text{ termina con la entrada } w\}.$$

Reducción. Definimos una función computable f tal que $\langle M, w \rangle \in A_{TM} \iff f(\langle M, w \rangle) \in Halt$.

Sobre la entrada $\langle M, w \rangle$, f construye la siguiente MT M' :

M' con cualquier entrada (ignorada) simula M con la entrada w :

- Si M acepta w , M' acepta (y termina).
- Si M rechaza w , M' entra en un loop infinito y *no* termina.

f devuelve la descripción $\langle M', \varepsilon \rangle$ (cualquier entrada sirve).

Verificación.

- Si $\langle M, w \rangle \in A_{TM}$ (es decir, M acepta w), entonces M' termina. Por lo tanto $\langle M', \varepsilon \rangle \in Halt$.
- Si $\langle M, w \rangle \notin A_{TM}$ (es decir, M no acepta w): puede pasar que M rechace o que no termine. En ambos casos M' *no* termina (en el primer caso por el loop, en el segundo por la simulación que no termina). Por lo tanto $\langle M', \varepsilon \rangle \notin Halt$.

Hemos exhibido f computable con $w \in A_{TM} \iff f(w) \in Halt$. Es decir, $A_{TM} \leq Halt$. Como A_{TM} es indecidible, $Halt$ también lo es. □

Observación 7.6. Comparará esta prueba con la primera prueba “informal” que dimos para H_{TM} al comienzo. La idea es la misma; lo que cambia es que ahora la formalizamos como una función computable f entre instancias, en lugar de describir una simulación que invoca al supuesto decididor de H_{TM} .

8. Resumen final

Las ideas centrales del teórico:

1. **Reducir un problema a otro** es la técnica fundamental para extender resultados de indecidibilidad. Si podemos transformar instancias de un problema indecidible A en instancias de B preservando las respuestas, entonces B es indecidible.
2. **Reducciones informales y formales.** Las reducciones se pueden expresar como “si tuviera un decodificador de B , decidiría A ” (forma informal con simulación), o como una función computable f con $w \in A \iff f(w) \in B$ (forma formal: $A \leq B$).
3. **Ejemplos clave de problemas indecidibles.**
 - H_{TM} (terminación) — reducción desde A_{TM} .
 - E_{TM} (vacío) — reducción desde A_{TM} con “máquina filtro”.
 - Reg (regularidad del lenguaje) — reducción usando $\{0^n 1^n\}$ como “testigo”.
 - Eq_{TM} (equivalencia) — reducción desde E_{TM} .
4. **Teorema de Rice.** Toda propiedad no trivial del lenguaje de una MT es indecidible. Es una herramienta para descartar de un saque la decidibilidad de un montón de problemas.
5. **Historias de ejecución.** Codificar la traza completa de una MT es útil para reducciones cuando hace falta “apoderarse” del paso a paso de la simulación.
6. **Autómatas linealmente acotados (LBAs).** Modelo intermedio. A_{LBA} es decidible (por la cota finita $|Q| \cdot n \cdot |\Gamma|^n$ de configuraciones). Sin embargo, E_{LBA} *sí* es indecidible.
7. **Problema de Correspondencia de Post (PCP).** Un problema combinatorio sobre fichas de dominó. Es indecidible mediante una reducción desde A_{TM} que codifica historias de ejecución. Es útil como “cabeza de puente” para probar indecidibilidad en problemas que no involucran MTs directamente.
8. **Propiedades fundamentales de $\leq$.**
 - Si $A \leq B$ y B es decidible, entonces A es decidible.
 - Si $A \leq B$ y B es reconocible, entonces A es reconocible.
 - Contrapositivas: si A es indecidible y $A \leq B$, entonces B es indecidible (idem para reconocibilidad).

Intuición

Las reducciones son el puente entre la teoría de la indecidibilidad y los problemas concretos que aparecen en la práctica. Una vez establecidos algunos pocos “núcleos duros” de indecidibilidad (A_{TM} , H_{TM} , PCP), todo el resto se demuestra encadenando reducciones. Más adelante en la materia veremos que la misma idea, restringida a reducciones *eficientes* (en tiempo polinomial), es la herramienta central de la teoría de la complejidad y de la noción de NP-completitud.